

CareFlow

Architecture Blueprint & System Diagrams

Technical documentation · generated from the live repository

28 June 2026

What this is. A high-level map of how CareFlow's existing parts fit together, plus ready-to-use Mermaid.js source for the system-architecture and entity-relationship diagrams. It was written from the actual codebase — the pnpm/Turborepo workspaces, packages/db/schema.sql, and the services/ + lib/supabase/ layers — so it matches the code as built.

Diagrams. Each diagram below appears as a rendered image followed by its Mermaid source. Paste any block into a ```mermaid fence on GitHub/GitLab, into mermaid.live, or the VS Code “Markdown Preview Mermaid” extension to reproduce or edit it.

Part 1 — Architecture Blueprint

1.1 The system at a glance

CareFlow is a **pnpm + Turborepo monorepo** with two deployable Next.js 16 apps and two shared packages. apps/web is the hospital-facing PWA that clinical and admin staff use; apps/owner-console is a separate platform-administration app (port 3001) for the CareFlow operator. Both depend on packages/shared (TypeScript domain types + Zod validation) and packages/db (schema.sql — the single source of truth for the database).

The backend is **Supabase (Postgres · Auth · Storage)**, so there is **no bespoke API server**. The browser talks to Supabase directly through the **anon client under Row-Level Security**; privileged work runs in Next.js Server Actions and the owner console through a **service-role client**. This keeps the moving parts few: a local-first client, a managed backend that also enforces access control, and a small server surface for the things a browser must not do.

Workspace	Type	Responsibility
@careflow/web	Next.js app	Hospital operations PWA — the clinical + admin workflow, local-first, offline-capable
@careflow/owner-console	Next.js app (:3001)	Platform operator console — cross-tenant oversight (suspend/reactivate, usage)
@careflow/db	Package	schema.sql — tables, RLS, functions, triggers, views (source of truth)
@careflow/shared	Package	Domain TypeScript types + Zod validation, imported by both apps

1.2 The local-first core (how the UI interacts with data)

The defining trait of apps/web is that it is **local-first**. The UI never blocks on the network: it reads and writes an in-browser cache (services/mockStorage.ts, persisted to localStorage), so every interaction is instant and the app keeps working through the power and connectivity gaps common in its target setting. Three modules turn that cache into a synchronized client:

- **Hydrate (download)** — supabaseData.hydrateFromSupabase() pulls the signed-in hospital's rows into the cache on sign-in, RLS-scoped and paged at 1000 rows per table.
- **Outbox (capture)** — every mutation is also recorded as a durable OutboxChange in services/syncQueue.ts (its own localStorage key), so it survives refreshes and offline use.
- **Drain (upload)** — the SyncEngine replays the outbox to Postgres in the background, emitting OUTBOX_EVENT (the sync-status chip) and CONFLICT_EVENT when a write loses an optimistic-concurrency check.

Telemetry rides the same client but is deliberately separate: services/telemetry.ts buffers metadata-only usage_events and flushes them fire-and-forget, never blocking clinical UX. The read/write/sync choreography is drawn in §2.2.

1.3 The backend (Supabase)

Supabase provides three managed services. **Auth** holds credentials and issues the session JWT. **Postgres** is the source of truth — 24 tables and three reporting views, plus functions and triggers — reached directly from the browser via the anon client, with Row-Level Security doing the access control the app would otherwise need an API tier for. **Storage** keeps clinical attachments (scanned results, patient documents) in two private buckets. The entire schema, including every policy, lives in packages/db/schema.sql.

1.4 Multi-tenancy & security model

CareFlow is **pooled multi-tenancy**: one Postgres database holds every hospital's data, and isolation is enforced by RLS rather than separate databases. Every domain table carries a hospital_id, and every policy ANDs in hospital_id = current_hospital_id() — a SECURITY DEFINER helper that resolves the caller's hospital from their single staff row. Reads are open to any active staff member of the tenant; writes are gated by role:

- **Doctors** — consultations, diagnoses, orders, prescriptions, admissions.
- **Nurses** — vitals (treatment_records), medication administration (MAR), care plans, transfers, allergies.
- **Lab techs** — results; **Pharmacists** — prescription status; **Receptionists/Admin** — patients, visits, billing.

Four mechanisms harden it: an append-only audit_log written by a SECURITY DEFINER trigger on every sensitive table (admin-readable, client-tamper-proof); **optimistic concurrency** (a version column bumped by trigger; the sync layer guards updates on the base version and surfaces conflicts); defense-in-depth CHECK constraints that mirror the app's Zod validation; and **tenant-prefixed storage** — every object path must start with the hospital_id, so files are isolated exactly like rows.

1.5 Authentication & onboarding

Staff sign in with a **username + password**, not email. Because Supabase Auth is email-keyed, each username maps to a synthetic, non-deliverable address (<username>@careflow.local); the staff identity travels in the user's metadata. Founding admins instead **verify identity first** — Google OAuth (PKCE, returning to /auth/callback) or a 6-digit email OTP — then call the `create_hospital_and_admin` SECURITY DEFINER RPC, which atomically creates the hospital (trial) and the founder's admin staff row and refuses a second hospital for the same owner. Provisioning additional staff logins needs the `service_role` key, so it runs only in the server action `app/actions/auth.ts`, never in the browser.

1.6 Platform / owner plane

A thin platform layer supports the operator without weakening tenant isolation. `apps/web` emits metadata-only `usage_events` (logins, registrations, feature use — never patient content) **write-only**: staff can insert events for their own hospital and read nothing back. `apps/owner-console` reads across tenants only through the **service-role client**, behind `requirePlatformAdmin()` — a server-side email allowlist (`PLATFORM_ADMIN_EMAILS`) that also records the admin in `platform_admins`. From there the operator can, for example, suspend or reactivate a hospital's subscription. `platform_admins` and cross-tenant reads are invisible to the normal authenticated/anon roles (RLS on, no policies — only the service role sees them).

1.7 Frontend composition

The hospital app uses the Next.js App Router with two route groups: (marketing) for the public landing, login, signup, onboarding and OAuth callback; and (app) for the authenticated workspace — dashboard (live journey board), intake, diagnostics, medications, care-plans, floor-map, departments, billing, reports, staff and reconciliation. It is bilingual (custom EN/FR i18n), themeable (light/dark via next-themes and semantic tokens), installable and offline-capable (web manifest + a service worker that precaches an app shell), charts with recharts, and exports reports via `jspdf` / `xlsx`.

1.8 Component interaction reference

Component	Responsibility	Talks to
<code>apps/web</code> · UI (App Router)	Renders the workflow screens; reads/writes the cache for instant UX	<code>mockStorage</code> · <code>syncQueue</code> · <code>supabaseAuth</code>
<code>services/mockStorage</code>	In-browser local-first cache (<code>localStorage</code>); the one store the UI reads	hydrated by <code>supabaseData</code> ; mutations → <code>syncQueue</code>
<code>services/supabaseData</code>	<code>hydrateFromSupabase()</code> : RLS-scoped download into the cache on sign-in	Supabase Postgres (anon)

Component	Responsibility	Talks to
services/syncQueue	Durable outbox + SyncEngine; optimistic concurrency; conflict/outbox events	Supabase Postgres (anon)
services/telemetry	Fire-and-forget, write-only usage metadata	Supabase Postgres (anon)
lib/supabase/client	Singleton anon browser client (PKCE session)	Supabase Auth/Postgres/Storage
lib/supabase/admin	Service-role client (server-only; bypasses RLS)	Supabase Auth/Postgres
lib/supabase/storage	Tenant-prefixed upload + signed download URLs	Supabase Storage (private buckets)
app/actions (auth · otp)	Server Actions: provision logins/hospital, request OTP	admin client · Resend
services/supabaseAuth	Username / Google / OTP sign-in; create_hospital_and_admin RPC	Supabase Auth/Postgres
apps/owner-console	Cross-tenant admin (SSR); suspend/reactivate; read telemetry	service-role · platform_admins · usage_events
packages/db (schema.sql)	Tables, RLS, functions, triggers, views — provisions Postgres	Supabase Postgres
packages/shared	Domain types + Zod validation	imported by both apps
manifest.ts · public/sw.js	Installable PWA + offline app shell	browser cache

1.9 Technology stack

Layer	Technology
Monorepo	pnpm workspaces + Turborepo
Frontend	Next.js 16.2 (App Router · Turbopack) · React 19.2
UI	shadcn + Base UI · Tailwind v4 · next-themes · lucide-react · recharts
Auth	Supabase Auth (@supabase/supabase-js; @supabase/ssr in owner-console)
Data	Supabase Postgres + Row-Level Security
Storage	Supabase Storage (private buckets, tenant-prefixed)
Validation	Zod (packages/shared)
Email	Resend (OTP / invitations)
Reporting / export	jspdf · jspdf-autotable · xlsx

Layer	Technology
i18n / PWA	custom EN/FR · web manifest + service worker (offline)
Testing	Vitest (+ RLS / onboarding / storage / concurrency integration suites)

1.10 Key data-flow narratives

1. **Read / hydrate** — sign-in → hydrateFromSupabase() → RLS-scoped SELECT * per table → cache replaced → UI renders from cache.
2. **Write / sync** — UI writes cache (instant) + enqueues outbox → SyncEngine drains → versioned UPDATEeq('version', base); stale ⇒ refetch + CONFLICT_EVENT; append-only ⇒ insert/upsert.
3. **File upload** — build <hospital_id>/../file path → upload to a private bucket (RLS checks the prefix) → store attachment_path → later read via a short-lived signed URL.
4. **Onboarding** — verify (Google/OTP) → create_hospital_and_admin RPC → hospital (trial) + founder admin row → staff provisioned later via the service-role server action.

Part 2 — System Diagrams (Mermaid.js)

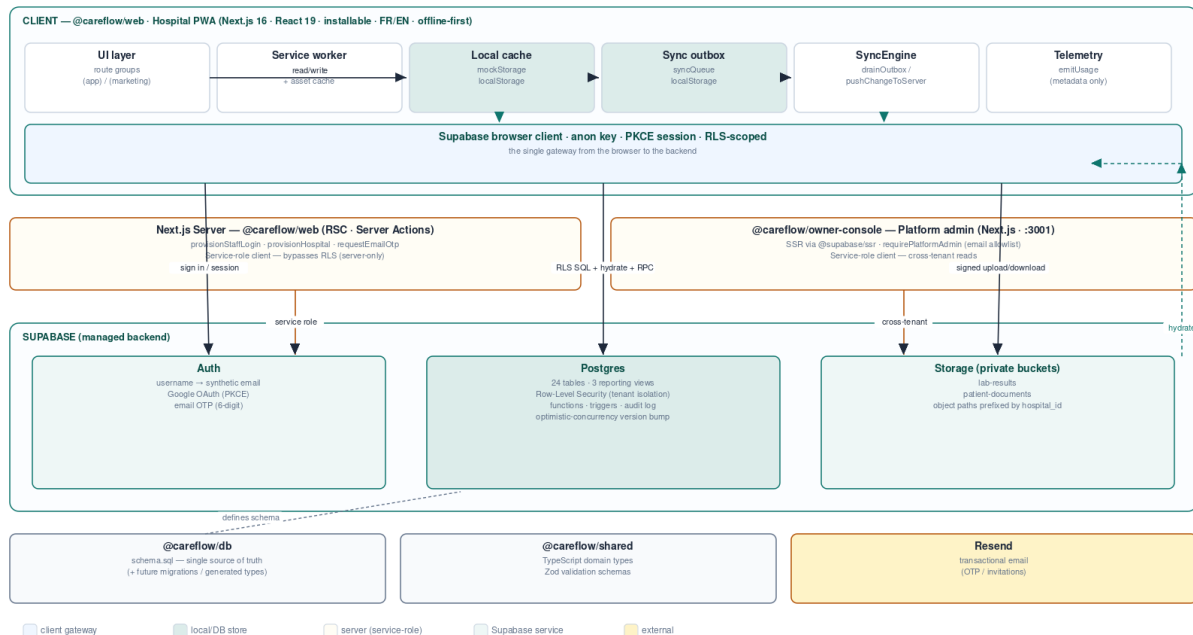
Four diagrams: the system architecture, the local-first sync sequence, and the data model at two levels of detail. Rendered image first, then the Mermaid source.

2.1 System Architecture Diagram

How the frontend, server actions, owner console, Supabase services, shared packages and the email provider connect. The browser is local-first; the anon client is the single gateway to the backend under RLS; the service-role paths (server actions, owner console) are the only cross-cutting, RLS-bypassing surfaces.

CareFlow — System Architecture

prpm + Turborepo monorepo · local-first PWA · Supabase backend



Mermaid source — flowchart (system architecture):

```
graph TD
    subgraph CLIENT ["CLIENT: Hospital PWA"]
        direction TB
        UI["UI layer<br/>route groups: (app) / (marketing)<br/>shadcn + Base UI - recharts - next-themes"]
        SW["Service worker<br/>offline shell + asset cache"]
        CACHE["Local-first cache<br/>mockStorage - localStorage"]
        OUTBOX["Sync outbox<br/>syncQueue - localStorage"]
        SYNC["SyncEngine<br/>drainOutbox / pushChangeToServer"]
        TELE["Telemetry buffer<br/>emitUsage (metadata only)"]
        ANON["Supabase browser client<br/>anon key - PKCE session"]
    end

    subgraph SERVER ["SERVER: web server actions"]
        direction TB
        ACT["Server Actions<br/>provisionStaffLogin - provisionHospital - requestEmailOtp"]
        ADMIN["Service-role client<br/>bypasses RLS - server-only"]
    end

    subgraph OWNER_CONSOLE ["OWNER CONSOLE: Platform admin (Next.js - port 3001)"]
        direction TB
        SSR["SSR via @supabase/ssr - requirePlatformAdmin (email admin)"]
        SRO["Service-role client - cross-tenant reads"]
        SUD["signed upload/download"]
    end

    subgraph SUPABASE ["SUPABASE (managed backend)"]
        direction TB
        AUTH["Auth<br/>username → synthetic email<br/>Google OAuth (PKCE)<br/>email OTP (6-digit)"]
        PG["Postgres<br/>24 tables · 3 reporting views<br/>Row-Level Security (tenant isolation)<br/>functions · triggers · audit log<br/>optimistic-concurrency version bump"]
        STG["Storage (private buckets)<br/>lab-results<br/>patient-documents<br/>object paths prefixed by hospital_id"]
    end

    subgraph SHARED ["@careflow/shared"]
        TS["TypeScript domain types<br/>Zod validation schemas"]
    end

    subgraph DB ["@careflow/db"]
        SC["schema.sql — single source of truth<br/>(+ future migrations / generated types)"]
    end

    subgraph RESEND ["Resend"]
        TE["transactional email (OTP / invitations)"]
    end

    CLIENT -- "service role" --> SUPABASE
    SERVER -- "service role" --> SUPABASE
    OWNER_CONSOLE -- "cross tenant" --> SUPABASE
    SUPABASE -- "hydrate" --> CLIENT
    SUPABASE -- "defines schema" --> DB
    SHARED -- "defines schema" --> DB
    RESEND -- "external" --> OWNER_CONSOLE
```

```

    OC["SSR pages - @supabase/ssr<br/>requirePlatformAdmin (email allowlist)"]
    OCADM["Service-role client<br/>cross-tenant reads"]
end

%% ===== SUPABASE BACKEND =====
subgraph SUPA["Supabase (managed backend)"]
    AUTH["Auth<br/>username to synthetic email<br/>Google OAuth - email OTP"]
    PG[("Postgres<br/>24 tables - RLS tenant isolation<br/>functions - triggers - views")]
    STOR[("Storage<br/>lab-results - patient-documents<br/>private - tenant-prefixed")]
end

%% ===== SHARED PACKAGES + EXTERNAL =====
subgraph PKG["Shared workspace packages"]
    DBP["@careflow/db<br/>schema.sql (source of truth)"]
    SH["@careflow/shared<br/>types + Zod validation"]
end
RESEND["Resend<br/>transactional email"]

%% ---- client internals ----
UI <-->|"instant read / write"| CACHE
UI -->|"mutations enqueue"| OUTBOX
SW -->|"serves shell offline"| UI
OUTBOX --> SYNC
UI --> TELE

%% ---- client to backend ----
ANON -->|"sign in / session"| AUTH
SYNC -->|"drain - upsert/insert/delete<br/>optimistic concurrency"| ANON
ANON -->|"RLS-scoped SQL"| PG
PG -->|"hydrateFromSupabase<br/>(RLS-scoped, paged)"| CACHE
ANON -->|"create_hospital_and_admin RPC"| PG
ANON -->|"signed upload / download"| STOR
TELE -->|"write-only usage_events"| ANON

%% ---- server actions ----
ACT --> ADMIN
ADMIN -->|"auth.admin.createUser"| AUTH
ADMIN -->|"service role"| PG
ACT -->|"OTP / invite email"| RESEND

%% ---- owner console ----
OC --> OCADM
OCADM -->|"cross-tenant: hospitals - usage_events - platform_admins"| PG

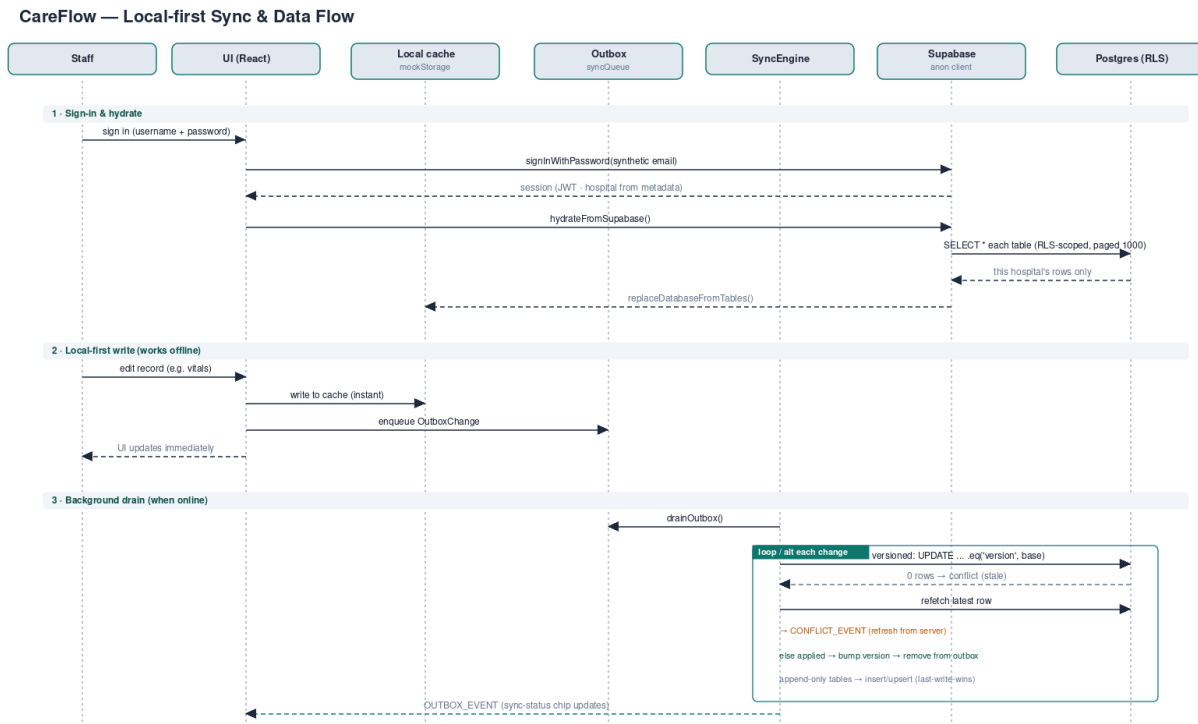
%% ---- packages ----
DBP -->|"defines schema"| PG
UI -->|"imports"| SH
OC -->|"imports"| SH

classDef store fill:#e2e8f0,stroke:#475569,color:#0f172a;
classDef ext fill:#fef9c3,stroke:#a16207,color:#0f172a;
class CACHE,OUTBOX,PG,STOR store;
class RESEND ext;

```

2.2 Local-first Sync & Data Flow (sequence)

The read/hydrate, offline write, and background-drain choreography — including the optimistic-concurrency conflict path that powers the sync-status chip.



Mermaid source — `sequenceDiagram` (sync & data flow):

```

sequenceDiagram
    autonumber
    actor Staff
    participant UI as UI (React)
    participant Cache as Local cache<br/>(mockStorage)
    participant Outbox as Outbox<br/>(syncQueue)
    participant Sync as SyncEngine
    participant SB as Supabase<br/>(anon client)
    participant PG as Postgres (RLS)

    Note over Staff,PG: Sign-in and hydrate
    Staff->>UI: sign in (username + password)
    UI->>SB: signInWithPassword (synthetic email)
    SB-->>UI: session (JWT, hospital from metadata)
    UI->>SB: hydrateFromSupabase()
    SB->>PG: SELECT * each table (RLS-scoped, paged 1000)
    PG-->>SB: this hospital's rows only
    SB-->>Cache: replaceDatabaseFromTables()

    Note over Staff,PG: Local-first write (works offline)
    Staff->>UI: edit record (e.g. vitals)
    UI->>Cache: write (instant)
    UI->>Outbox: enqueue OutboxChange
    Outbox-->>Staff: UI updates immediately

    Note over Outbox,PG: Background drain when online
    Sync->>Outbox: drainOutbox()
    loop each pending change
        alt versioned table (optimistic concurrency)
            Sync->>PG: update ... .eq('version', base)
            alt 0 rows matched (stale version)
                PG-->>Sync: conflict
            else
                PG-->>Sync: CONFLICT_EVENT (refresh from server)
                Sync-->>Outbox: else applied -> bump version -> remove from outbox
                Sync-->>PG: append-only tables -> insert/upsert (last-write wins)
            end
        end
    end
    Outbox-->>UI: OUTBOX_EVENT (sync-status chip updates)
  
```

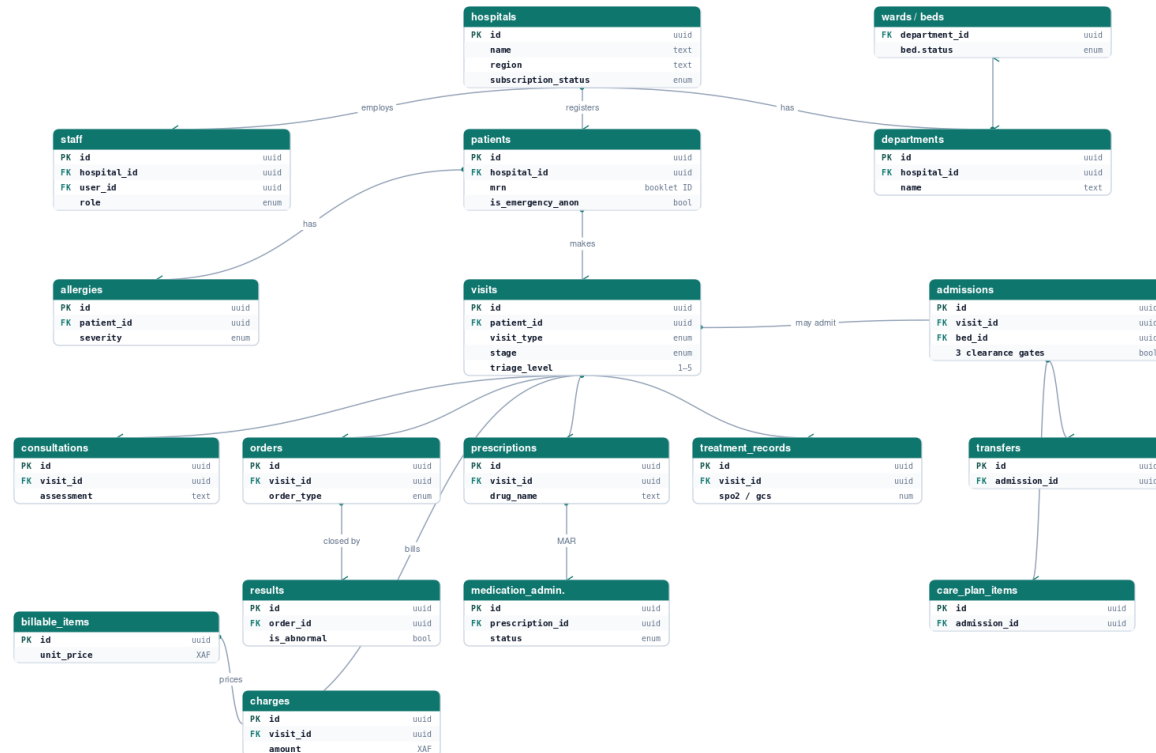
```
    Sync->>PG: refetch latest row
    Sync->>UI: CONFLICT_EVENT (refresh from server)
else applied
    PG-->>Sync: bumped version
    Sync->>Outbox: remove change
end
else append-only clinical table
    Sync->>PG: insert / upsert (last-write-wins)
    PG-->>Sync: ok
    Sync->>Outbox: remove change
end
end
Sync-->>UI: OUTBOX_EVENT (sync-status chip updates)
```

2.3 Entity-Relationship Diagram — Overview

The readable spine of the data model. The **visit** is the centre of the record; clinical detail, admission, and billing all hang off it. Every domain table also carries `hospital_id` → hospitals (tenant isolation), omitted here for clarity.

CareFlow — Data Model (Overview)

The visit is the spine of the record. Every domain table also carries `hospital_id` → hospitals (tenant isolation).



Mermaid source — erDiagram (overview):

```
erDiagram
    hospitals ||--o{ departments : "has"
    hospitals ||--o{ staff : "employs"
    hospitals ||--o{ patients : "registers"
    departments ||--o{ wards : "contains"
    wards ||--o{ beds : "holds"
    patients ||--o{ allergies : "has"
    patients ||--o{ visits : "makes"
    visits ||--o{ consultations : "documents"
    visits ||--o{ diagnoses : "yields"
    visits ||--o{ orders : "requests"
    orders ||--o{ results : "closed by"
    visits ||--o{ prescriptions : "prescribes"
    prescriptions ||--o{ medication_administrations : "given as"
    visits ||--o{ treatment_records : "vitals"
    visits ||--o{ admissions : "may admit"
    admissions ||--o{ transfers : "moves via"
    admissions ||--o{ care_plan_items : "plans"
    care_plan_items ||--o{ care_plan_entries : "logged as"
    admissions }o--|| wards : "placed in"
    admissions }o--o| beds : "occupies"
    visits ||--o{ charges : "bills"
    billable_items ||--o{ charges : "prices"

    hospitals {
        uuid id PK
        text name
    }
    departments {
        uuid id PK
        uuid hospital_id FK
        text name
    }
    wards_beds {
        uuid id PK
        uuid department_id FK
        enum bed_status
    }
    patients {
        uuid id PK
        uuid hospital_id FK
        text mrn
        bool is_emergency_anon
    }
    allergies {
        uuid id PK
        uuid patient_id FK
        enum severity
    }
    visits {
        uuid id PK
        uuid patient_id FK
        enum visit_type
        enum stage
        enum triage_level
    }
    consultations {
        uuid id PK
        uuid visit_id FK
        text assessment
    }
    orders {
        uuid id PK
        uuid visit_id FK
        enum order_type
    }
    prescriptions {
        uuid id PK
        uuid visit_id FK
        text drug_name
    }
    treatment_records {
        uuid id PK
        uuid visit_id FK
        num spo2_gcs
    }
    results {
        uuid id PK
        uuid order_id FK
        bool is_abnormal
    }
    medication_admin {
        uuid id PK
        uuid prescription_id FK
        enum status
    }
    transfers {
        uuid id PK
        uuid admission_id FK
    }
    care_plan_items {
        uuid id PK
        uuid admission_id FK
    }
    billable_items {
        uuid id PK
        float unit_price XAF
    }
    charges {
        uuid id PK
        uuid visit_id FK
        float amount XAF
    }
```

```

    text region
    enum subscription_status
    jsonb feature_flags
  }
  staff {
    uuid id PK
    uuid hospital_id FK
    uuid user_id FK "auth.users"
    enum role
    text full_name
  }
  patients {
    uuid id PK
    uuid hospital_id FK
    text mrn "Cameroon booklet ID"
    text full_name
    bool is_emergency_anonymous
  }
  visits {
    uuid id PK
    uuid hospital_id FK
    uuid patient_id FK
    enum visit_type
    enum stage "care journey"
    smallint triage_level
  }
  admissions {
    uuid id PK
    uuid visit_id FK
    uuid bed_id FK
    bool is_medical_cleared
    bool is_financial_cleared
    bool is_pharmacy_ready
  }
  charges {
    uuid id PK
    uuid visit_id FK
    enum source
    int amount "XAF"
    enum status
  }
}

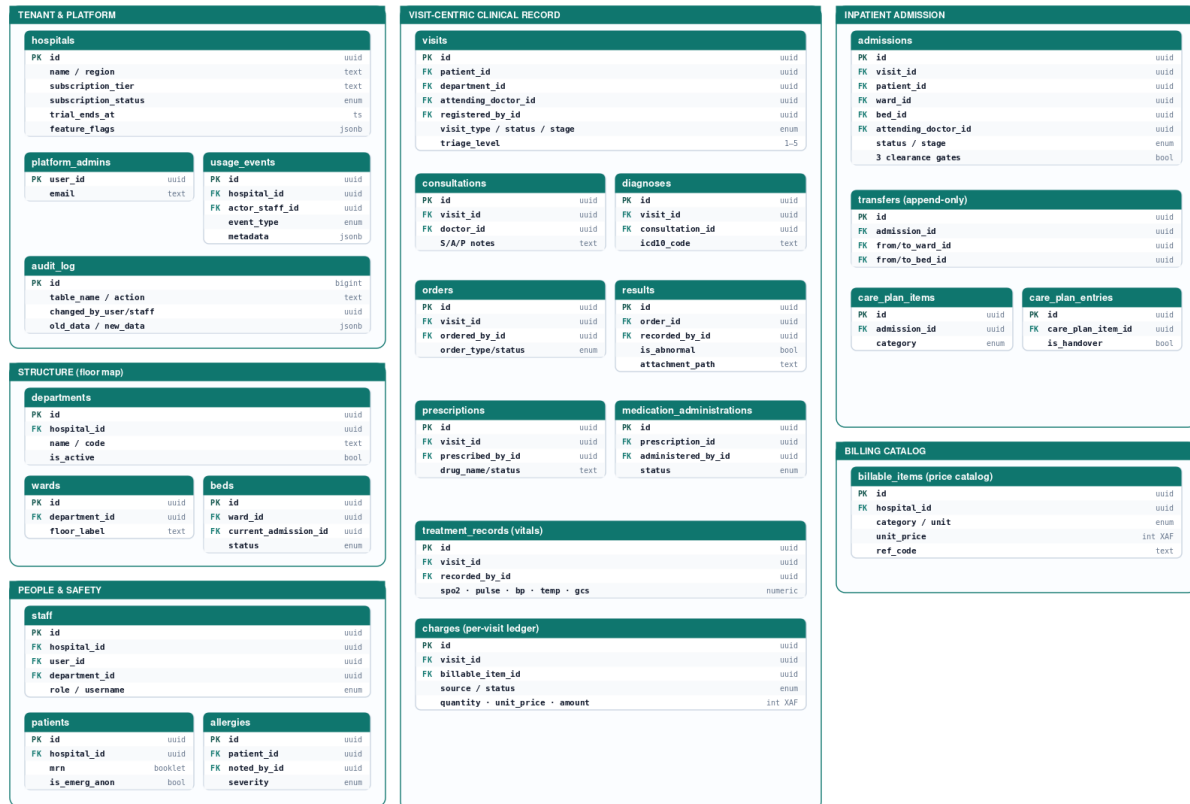
```

2.4 Entity-Relationship Diagram — Full reference (24 tables)

Every table with its key columns (**PK** primary key, **FK** foreign key), grouped by domain. The Mermaid source below is the exact, complete relationship set. Note: every domain table carries `hospital_id` → `hospitals` (RLS); `staff` is the actor referenced by the `*_by_id` columns — listed as columns and, in the code, only the structural edges are drawn to keep the graph legible.

CareFlow — Data Model (Full Reference · 24 tables)

Key columns shown (PK = primary key, FK = foreign key). Relationships: see the Mermaid ERD code. Every domain table carries `hospital_id` → `hospitals` (RLS); `staff` is the actor on `*_by_id` columns.



Mermaid source — erDiagram (full reference):

```
erDiagram
    %% ===== Tenant root (every domain table also carries hospital_id FK -> hospitals) =====
    auth_users ||--o| staff : "authenticates"
    auth_users ||--o| platform_admins : "authorizes"
    hospitals ||--o| departments : ""
    hospitals ||--o{ staff : ""
    hospitals ||--o{ patients : ""
    hospitals ||--o{ visits : ""
    hospitals ||--o{ billable_items : ""
    hospitals ||--o{ usage_events : ""

    %% ===== Structure / floor map =====
    departments ||--o{ wards : ""
    departments ||--o{ staff : "assigned"
    wards ||--o{ beds : ""

    %% ===== People / safety =====
    patients ||--o{ allergies : ""
    patients ||--o{ visits : ""
    patients ||--o{ admissions : ""

    %% ===== Visit-centric clinical record =====
```

```

visits      ||--o{ consultations : ""
visits      ||--o{ diagnoses      : ""
consultations ||--o{ diagnoses      : ""
visits      ||--o{ orders          : ""
orders       ||--o{ results         : ""
visits      ||--o{ prescriptions   : ""
prescriptions ||--o{ medication_administrations : ""
visits      ||--o{ treatment_records : ""
visits      ||--o{ admissions       : ""
visits      ||--o{ charges          : ""

%% ===== Inpatient admission =====
admissions  ||--o{ transfers        : ""
admissions  ||--o{ care_plan_items : ""
admissions  ||--o{ care_plan_entries : ""
care_plan_items ||--o{ care_plan_entries : ""
admissions  }o--|| wards            : "ward"
admissions  }o--o| beds             : "bed"

%% ===== Billing =====
billable_items ||--o{ charges      : ""

%% =====
hospitals {
  uuid id PK
  text name
  text region
  text subscription_tier
  enum subscription_status
  ts trial_ends_at
  jsonb feature_flags
}
departments {
  uuid id PK
  uuid hospital_id FK
  text name
  text code
  bool is_active
}
wards {
  uuid id PK
  uuid hospital_id FK
  uuid department_id FK
  text name
  text floor_label
}
beds {
  uuid id PK
  uuid hospital_id FK
  uuid ward_id FK
  uuid current_admission_id FK
  text label
  enum status
}
staff {
  uuid id PK
  uuid hospital_id FK
  uuid user_id FK "auth.users"
  uuid department_id FK
  text full_name
  enum role
  text username
}
patients {
  uuid id PK
  uuid hospital_id FK
  text mrn "booklet ID"
  text full_name
  enum sex
  bool is_emergency_anonymous
  bool no_known_allergies
}

```

```

allergies {
  uuid id PK
  uuid hospital_id FK
  uuid patient_id FK
  uuid noted_by_id FK "staff"
  text substance
  enum category
  enum severity
}
visits {
  uuid id PK
  uuid hospital_id FK
  uuid patient_id FK
  uuid department_id FK
  uuid attending_doctor_id FK "staff"
  uuid registered_by_id FK "staff"
  enum visit_type
  enum status
  enum stage
  smallint triage_level
}
consultations {
  uuid id PK
  uuid hospital_id FK
  uuid visit_id FK
  uuid doctor_id FK "staff"
  text subjective
  text assessment
  text plan
}
diagnoses {
  uuid id PK
  uuid hospital_id FK
  uuid visit_id FK
  uuid consultation_id FK
  uuid diagnosed_by_id FK "staff"
  text icd10_code
  bool is_primary
}
orders {
  uuid id PK
  uuid hospital_id FK
  uuid visit_id FK
  uuid ordered_by_id FK "staff"
  enum order_type
  enum status
}
results {
  uuid id PK
  uuid hospital_id FK
  uuid order_id FK
  uuid recorded_by_id FK "staff"
  text value
  bool is_abnormal
  text attachment_path
}
prescriptions {
  uuid id PK
  uuid hospital_id FK
  uuid visit_id FK
  uuid prescribed_by_id FK "staff"
  text drug_name
  enum status
}
medication_administrations {
  uuid id PK
  uuid hospital_id FK
  uuid prescription_id FK
  uuid administered_by_id FK "staff"
  enum status
  ts scheduled_for
}
treatment_records {
  uuid id PK

```

```

    uuid hospital_id FK
    uuid visit_id FK
    uuid recorded_by_id FK "staff"
    numeric spo2
    numeric pulse
    int gcs_score
}
admissions {
    uuid id PK
    uuid hospital_id FK
    uuid visit_id FK
    uuid patient_id FK
    uuid ward_id FK
    uuid bed_id FK
    uuid attending_doctor_id FK "staff"
    enum status
    enum stage
    bool is_medical_cleared
    bool is_financial_cleared
    bool is_pharmacy_ready
}
transfers {
    uuid id PK
    uuid hospital_id FK
    uuid admission_id FK
    uuid patient_id FK
    uuid from_ward_id FK
    uuid to_ward_id FK
    uuid from_bed_id FK
    uuid to_bed_id FK
}
care_plan_items {
    uuid id PK
    uuid hospital_id FK
    uuid admission_id FK
    uuid patient_id FK
    enum category
    enum status
    text goal
}
care_plan_entries {
    uuid id PK
    uuid hospital_id FK
    uuid admission_id FK
    uuid care_plan_item_id FK
    text note
    bool is_handover
}
billable_items {
    uuid id PK
    uuid hospital_id FK
    enum category
    text name
    enum unit
    int unit_price "XAF"
    text ref_code
}
charges {
    uuid id PK
    uuid hospital_id FK
    uuid visit_id FK
    uuid billable_item_id FK
    enum source
    text source_ref_id
    int quantity
    int amount "XAF"
    enum status
}
audit_log {
    bigint id PK
    text table_name
    text record_id
    text action
    uuid changed_by_user

```

```
  uuid changed_by_staff
  uuid hospital_id
  jsonb old_data
  jsonb new_data
}
usage_events {
  uuid id PK
  uuid hospital_id FK
  uuid actor_staff_id FK "staff"
  enum event_type
  jsonb metadata
}
platform_admins {
  uuid user_id PK "auth.users"
  text email
  text full_name
}
auth_users {
  uuid id PK "Supabase Auth"
  text email
}
```

2.5 Rendering the Mermaid source

- **GitHub / GitLab** — paste a block inside a ````mermaid` fenced code block in any .md file; it renders automatically.
- **mermaid.live** — the official live editor; paste, preview, and export SVG/PNG.
- **VS Code** — the “Markdown Preview Mermaid Support” extension renders it in preview.
- **CLI** — `npx @mermaid-js/mermaid-cli -i diagram.mmd -o diagram.svg` for scripted exports.

Appendix — Source-of-truth map & notes

Where each thing lives in the repo

Concern	File / path
Database schema (ERD source)	packages/db/schema.sql
Domain types	packages/shared/types/healthcare.ts
Validation (Zod)	packages/shared/validation/*
Local cache	apps/web/services/mockStorage.ts
Hydrate (download)	apps/web/services/supabaseData.ts
Sync outbox (upload)	apps/web/services/syncQueue.ts
Telemetry	apps/web/services/telemetry.ts
Auth (browser)	apps/web/services/supabaseAuth.ts
Supabase clients / storage	apps/web/lib/supabase/{client,admin,identity,storage}.ts
Server Actions	apps/web/app/actions/{auth,otp}.ts
Routes	apps/web/app/(app)/* · apps/web/app/(marketing)/*
Owner console	apps/owner-console/*
PWA / offline	apps/web/app/manifest.ts · apps/web/public/sw.js

Notes & assumptions

- Reflects the repository post-monorepo migration, including the Phase 19.1 telemetry tables (usage_events, platform_admins).
- The ERDs show key columns; every domain table also has created_at/updated_at, and mutable tables a version column — the complete column list is in schema.sql.
- “Service” here means the service-layer modules (services/, lib/supabase/), not separate microservices — the architecture is one Next.js app + Supabase + a separate owner-console app.